7 - Data Scientist

FINANCIAL DATA

—

Python

CODECAMY

Welcome to Analyze Financial Data with Python

An introduction to the Analyze Financial Data with Python Skill Path.

Welcome to Analyze Financial Data with Python! We're excited to start this journey with you.

Analyze Financial Data with Python is a Skill Path that will teach you the fundamentals of programming in Python, along with how to process, analyze, and visualize financial data. As you work through this path, you'll take Codecademy lessons, complete practice projects, and investigate actual financial data both right here on Codecademy and on your own computer. This material will prepare you to:

- Interpret and calculate financial statistics
- Import financial data from APIs
- Optimize a portfolio of assets
- Communicate data-driven conclusions about investments

To start, you will begin with introductory Python programming, as well as discovering why Python is a useful tool in Finance. Before wrapping up this section, you will learn how to use Jupyter Notebooks, a useful tool for data analysis, and dive into calculating financial statistics.

Let's get started!

Calculating Net Present Value

Frances is working with her team to evaluate two projects they are thinking about pursuing, but they can't do both. Frances needs to help her team decide how to allocate their investment.

Calculating the Net Present Value (NPV) will help us analyze the profitability of these two projects by giving us a present value of the future cash inflows and outflows over the life of both projects.

$$NPV=\sum_{t=0}^{n}CFt(1+r)t$$

$$NPV=\sum_{t=0}^{n}\frac{CF_t}{(1+r)^t}$$

Python can easily calculate the NPV of a set of cash flows. Since any NPV calculation is dependent on r, the rate at which we are discounting future cash flows to the present, it can be useful to calculate different NPVs at different discount rates, and build an NPV profile to see how the NPV of a project changes based upon the discount rate applied to the calculation.

Frances is asked to create NPV profiles for two projects her team is evaluating to determine which project they should choose. SuperBank has $1,000,000 available to invest, and two clean energy projects available for investment that will generate annual cash flows over 20 years. By building and graphing an NPV Profile, she can easily see how discount rates affect both projects and help her team make a good decision.

Project A is represented by the red line on the chart, and Project B is represented by the blue line.

Basic Stock Analysis

Frances has now been asked to help support a new investment initiative. Frances needs to perform analysis on Apple (NASDAQ: AAPL) as part of a broader project her team has undertaken to support a new opportunity.

First, Frances has decided that she wants to better understand the price movements of AAPL as part of her research. She decides to use pandas and numpy to import and calculate the information she needs, and matplotlib to graph her results.

Frances decides to build a histogram of the daily rates of
Preview: Docs Loading link description
return
for a 6 month period, and calculate the standard deviation of those returns to get a better understanding of the historical volatility of the stock during that 6 month period.

```
import codecademylib3_seaborn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('AAPL_data.csv')
print(df.head())

df['Daily Log Rate of Return'] = np.log(df['Adj Close']/df['Adj
Close'].shift(1))

print(df['Daily Log Rate of Return'])

stdev = np.std(df['Daily Log Rate of Return'])
print(stdev)

plt.hist(df['Daily Log Rate of Return'].dropna())
plt.title('Histogram of AAPL Daily Log Rates of Return')
plt.xlabel('Log Rate of Return')
plt.ylabel('Number of Days')
plt.show()
```

Python Candlestick Chart

Frances is asked to build a candlestick chart for her team to help everyone get an idea of how the daily pricing of AAPL changed over time, and what the price movements looked like during each day of trading.

A candlestick chart plots the daily opening price, closing price, lowest price and highest price of a particular stock, and shows how that price changed each day over a given period of time. Each day is shown by one “candlestick,” and can be helpful when analysts are trying to make predictions about how a particular price of a stock may move in the future.

```
import codecademylib3_seaborn
import datetime
import pandas as pd
```

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from mpl_finance import candlestick_ohlc

df = pd.read_csv('AAPL_data.csv')
print(df.head())

df['Date'] = pd.to_datetime(df['Date'])
df["Date"] = df["Date"].apply(mdates.date2num)

candle_data = df[['Date', 'Open', 'High', 'Low', 'Close']]
print(candle_data.head())

f1, ax = plt.subplots(figsize = (10,5))
candlestick ohlc(ax,candle data.values, colorup='green', colordown='red')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
plt.title('Candlestick Chart for AAPL')
plt.xlabel('Date')
plt.ylabel('Value($)')
plt.show()
```

Calculating the Efficient Frontier

After feeling comfortable with her research, Frances needs to build a model portfolio with a basket of stocks, including AAPL, to present some recommendations to her team. She decides to construct her model portfolio with AAPL, Ford (NYSE: F), Dollar General (NYSE: DG), JetBlue (NASDAQ: JBLU) and Starbucks (NASDAQ: SBUX).

Instead of going to a website to download information into csv format and using Excel to create her graphs, she decides to import stock information with quandl, and use pandas and numpy to perform calculations on her data, and matplotlib to plot her results. Frances will construct an “efficient frontier” to visualize the optimal portfolios of the five stocks she is looking into.

```
import pandas as pd
import matplotlib.pyplot as plt
from rf import return_portfolios, optimal_portfolio
import codecademylib3_seaborn
import numpy as np

path='quarters.csv'
stock_data = pd.read_csv(path)
print(stock_data.head())
selected=list(stock_data.columns[1:])

returns_quarterly = stock_data[selected].pct_change()
expected_returns = returns_quarterly.mean()
cov_quarterly = returns_quarterly.cov()

random_portfolios = return_portfolios(expected_returns, cov_quarterly)

weights, returns, risks = optimal_portfolio(returns_quarterly[1:])

random_portfolios.plot.scatter(x='Volatility', y='Returns', fontsize=12)
try:
  plt.plot(risks, returns, 'y-o')
except:
```

```
  pass
plt.ylabel('Expected Returns',fontsize=14)
plt.xlabel('Volatility (Std. Deviation)',fontsize=14)
plt.title('Efficient Frontier', fontsize=24)
plt.show()
```

Python for Finance: Summary

Frances has been enjoying her work as a financial analyst for SuperBank, and feels confident that she has the tools she needs to support her team and provide her colleagues and her bosses with valuable insight to drive business and investment decisions.

With Python, she can step up her analysis and build powerful models that:

Retrieve and analyze information using Python libraries that make it easy to access and transform data from various sources;
Write code that allows her to easily perform calculations on the data she needs to analyze
Graph her results using matplotlib to visualize her analysis and share her results for improved decision making.

Lambda Functions

Learn about Lambda Functions in Python!
Python is known for its simplicity and readability, and one of its powerful features is the ability to create functions. While you're probably familiar with defining functions using the `def` keyword, Python also offers a more concise way to create small, anonymous functions called lambda functions.

In this article, we'll dive into the world of lambda functions, exploring their syntax, use cases, and best practices. By the end, you'll have a solid understanding of how to leverage these compact functions in your Python code.

What are Lambda Functions?

Lambda functions, also known as anonymous functions, are small, inline functions that can have any number of arguments but only one expression. They are defined using the `lambda` keyword and are typically used for short, simple operations.

Unlike regular functions defined with `def`, lambda functions don't have a name and are usually used in situations where you need a simple function for a short period of time.

Let's compare a regular function with a lambda function:

```
# Regular function

def square(x):
```

```
    return x ** 2


# Lambda function
square_lambda = lambda x: x ** 2
```

Both functions square the input, but the lambda function is more concise.

Syntax of Lambda Functions

The basic syntax of a lambda function is:

```
lambda [arguments]: [expression]
```

Here are a couple of simple examples:

```
# Lambda function to add two numbers
add = lambda a, b: a + b
print(add(3, 5))  # Output: 8


# Lambda function to print a name
greeting = lambda name: f"Hello, {name}!"
print(greeting("Alice"))  # Output: Hello, Alice!
```

Using Lambda Functions

Lambda functions are most commonly used as arguments to higher-order functions such as `map()`, `filter()`, and `sorted()`. Higher-order functions are functions that can accept other functions, such as lambda functions, as arguments. Let's look at some examples:

Using Lambda with `map()`

The `map()` function applies the given lambda function to each item in a list:

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared)  # Output: [1, 4, 9, 16, 25]
```

In this example, the lambda function `lambda x: x ** 2` squares each number in the `numbers` list. The `map()` function applies this lambda to each element, resulting in a new list where every number is squared.

Using Lambda with `filter()`

The `filter()` function creates a new list of elements for which the given lambda function returns True:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers)  # Output: [2, 4, 6, 8, 10]
```

Here, the lambda function `lambda x: x % 2 == 0` checks if a number is even. The `filter()` function uses this lambda to keep only the even numbers from the original list, creating a new list containing only even numbers.

Using Lambda with `sorted()`

The `sorted()` function can use a lambda function as a key for custom sorting:

```
students = [('Alice', 'A', 15), ('Bob', 'B', 12), ('Charlie', 'A', 20)]

sorted_students = sorted(students, key=lambda x: x[2])

print(sorted_students)

# Output: [('Bob', 'B', 12), ('Alice', 'A', 15), ('Charlie', 'A', 20)]
```

In this case, the lambda function `lambda x: x[2]` is used as the key for sorting. It tells the `sorted()` function to use the third element (index 2) of each tuple for comparison. As a result, the list of students is sorted based on their age (the third element in each tuple).

We'll further discuss the higher-order functions, such as the `map()` function, in more detail in our next article.

Advantages and Limitations

Lambda functions offer several advantages:

However, they also have limitations:

Best Practices

Use lambda functions when:

Avoid lambda functions when:

When lambda functions become too complex, it's often better to use a regular function defined with `def`. This improves readability and makes your code easier to maintain.

Wrapping Up

Lambda functions are a powerful feature in Python that allow you to write more concise and functional code. They're particularly useful for simple operations and as arguments to higher-order functions. However, it's important to use them judiciously and switch to regular functions when the logic becomes more complex.

Practice using lambda functions in your code to become more comfortable with them. Try rewriting some of your existing functions as lambda functions where appropriate, and experiment with using them in `map()`, `filter()`, and `sorted()`.

Map Functions

Learn about Map Functions in Python!
Python provides several built-in functions that help with functional programming. One of the most versatile and commonly used is the `map()` function. In this article, we'll explore what `map()` is, how it works, and when to use it in our Python programs.

What is the map() Function?

The `map()` function is a built-in function in Python that applies a given function to each item in an iterable (like a list, tuple, or string) and returns a new iterable as a result.

The basic syntax of `map()` is:

```
map(function, iterable, [iterable2, iterable3, ...])
```

```
function
iterable
```

How map() Works

Let's break down how `map()` works with a simple example:

```
def double(x):
    return x * 2


numbers = [1, 2, 3, 4, 5]
doubled_numbers = map(double, numbers)
```

```
print(list(doubled_numbers))  # Output: [2, 4, 6, 8, 10]
```

In this example, `map()` applies the `double` function to each number in the `numbers` list, creating a new iterator with the results.

Using map() with Built-in Functions

We can use `map()` with Python's built-in functions. For example, we can quickly convert a list of strings to integers:

```
# Converting strings to integers

str_nums = ['1', '2', '3', '4', '5']

int_nums = list(map(int, str_nums))

print(int_nums)  # Output: [1, 2, 3, 4, 5]
```

Or find the length of another list of strings:

```
# Finding the length of strings

words = ['apple', 'banana', 'cherry']

word_lengths = list(map(len, words))

print(word_lengths)  # Output: [5, 6, 6]
```

Using map() with Lambda Functions

But `map()` is most often used in conjunction with lambda functions for quick, one-off operations. For example, here we easily double each number in a list:

```
numbers = [1, 2, 3, 4, 5]

doubled = list(map(lambda x: x * 2, numbers))

print(doubled)  # Output: [2, 4, 6, 8, 10]
```

Multiple Iterables with map()

`map()` can also work with multiple iterables, assuming the function can take multiple inputs:

```
list1 = [1, 2, 3]

list2 = [10, 20, 30]


result = list(map(lambda x, y: x + y, list1, list2))

print(result)  # Output: [11, 22, 33]
```

Advantages of Using map()

Readability: `map()` can make our code more concise and easier to read, especially for simple operations on iterables.
Memory Efficiency: `map()` returns an iterator, which means it doesn't create the entire result list in memory at once. This can be beneficial when working with large datasets.
Functional Programming: `map()` encourages a functional programming style, which can lead to more maintainable and testable code.

When to Use map()

Use `map()` when:

We need to apply a function to every item in an iterable.
We want to transform data without explicitly writing a for loop.
We're working with large datasets and want to avoid creating intermediate lists.

Alternatives to map()

While `map()` is powerful, there also are alternatives:

List Comprehensions: Often more readable for simple operations.

```
doubled = [x * 2 for x in numbers]
```

Generator Expressions: Similar to list comprehensions but return an iterator.

```
doubled = (x * 2 for x in numbers)
```

For Loops: More verbose but sometimes clearer for complex operations.

```
doubled = []
 for x in numbers:
     doubled.append(x * 2)
```

Best Practices

Use `map()` when it makes your code more readable and concise.
For complex operations, consider using a regular function instead of a lambda.
Remember that `map()` returns an iterator. If you need a list, wrap it with `list()`.
When working with multiple iterables, ensure they have the same length to avoid unexpected results.

Wrapping Up

The `map()` function is a powerful tool in Python for applying a function to every item in an iterable. It's particularly useful for data transformation tasks and can lead to more concise and functional code. However, like any tool, it's important to use it judiciously. Sometimes a list comprehension or a simple for loop might be more appropriate.

Practice using `map()` in your Python programs. Try rewriting some of your existing loops as `map()` operations, and experiment with different functions and iterables. As you gain more experience, you'll develop a better sense of when `map()` is the right tool for the job.

Match Statements in Python

Learn how to declare and use match-case statements in Python.
Like the if-else statements, we can also use switch statements to build a program's control flow. In Python, we have match-case statements that implement switch statements.

Let's discuss switch and match statements, how to define them in a Python program, and when to use match-case statements instead of if-else statements to build control flow in a program.

What is a Switch Statement?

A switch statement is a control flow statement that executes a block of code, among many others, based on the value of a variable or expression.

Functionally, switch-case statements are almost like if-else statements. For example, let's look at the code from the if statements exercise.

```
user_name = "Dave"
if user_name == "Dave":
    print("Get off my computer Dave!")
elif user_name == "angela_catlady_87":
    print("I know it is you, Dave! Go away!")
elif user_name == "Codecademy":
    print("Access Granted.")
else:
    print("Username not recognized.")
```

The above code can be written using a switch statement, as shown in the following code. Don't worry about the syntax. It's explained in the next section.

```
user_name = "Dave"
switch user_name:
    case "Dave":
        print("Get off my computer Dave!")
    case "angela_catlady_87":
        print("I know it is you, Dave! Go away!")
    case "Codecademy":
        print("Access Granted.")
    case default:
        print("Username not recognized.")
```

If you run the above code, it will encounter a `SyntaxError` because Python does not have a switch keyword. Python didn't have switch statements or any of its implementations until version 3.10.

In Python 3.10, we got match-case statements that implement switch statements and other functionalities. From now on, we will refer to switch-case statements as match-case statements, as defined in Python.

How to Declare a Match Statement in Python?

We use the `match`, `case`, and `default` keywords to define match-case statements in Python. It has the following syntax:

```
match expression:
    case value_1:
        # code to execute when expression equals value_1
    case value_2:
        # code to execute when expression equals value_2
    case value_3:
        # code to execute when expression equals value_3
    case value_4:
        # code to execute when expression equals value_4
    case value_N:
        # code to execute when expression equals value_N
    case default:
        # code to execute when expression isn't equal to any of the values
```

In the above code,

`match` is a keyword that marks the start of the match block.
`expression` can be a variable, an arithmetic expression, a Boolean expression, or a Python object like a list, tuple, or string.
The code inside a case block is executed for a single value of expression. For example, only if `expression` evaluates to `value_1` will the code inside the `case value_1` block be executed. If `expression` evaluates to `value_2`, then only the code inside the `case value_2` block will be executed.
Only one case block is executed in the entire match block for any given value of `expression`. Once a case block is executed, the program's control flow moves out of the match block.
The `default` block is always present at the end of the match block. If `expression` matches none of the values in other case blocks, the code inside `case default` is executed.

Using the above syntax, we can rewrite the code given in the switch-case example as shown below:

```
user_name = "Dave"
match user_name:
    case "Dave":
        print("Get off my computer Dave!")
    case "angela_catlady_87":
        print("I know it is you, Dave! Go away!")
    case "Codecademy":
        print("Access Granted.")
    case default:
        print("Username not recognized.")
```

If you run the above code, you will get the following output:

```
Get off my computer Dave!
```

When to Use Match Statements in Python?

We can use match-case statements for the following use cases:

Match-case statements can be an efficient alternative to if-elif-else statements. When a variable or expression can take multiple values, and we need to perform a different action for each possible value, we can use match statements.
A match-case block can be used for other tasks, like structural pattern matching, in addition to replacing if-else blocks. This helps us check for different conditions on Python objects like lists, tuples, and strings.
Match-case statements are more readable compared to if-elif-else statements. Hence, in cases where we must check for many values that an expression or variable can take, we should use match-case statements.

Simple Rate of Return

Now that we are familiar with how the rate of

Preview: Docs Loading link description
return
is usually expressed, let's take a look at calculating it.
The most basic type of return is the *simple rate of return*. It is defined as the difference between the starting and ending price of an investment over a time period, divided by the starting price. If an investment returns dividends, those dividends should be added to the numerator.

$$R = (E - S + D) / S$$

- `R`: simple rate of return
- `S`: starting price of investment
- `E`: ending price of investment
- `D`: dividend

In the equation above, the numerator represents the absolute amount gained or lost in an investment. For example, if the starting price of an investment is `$25` and the ending price is `$30`, then there is a `$5` profit.

While this gives us some information about the return of an investment, it is by itself not very useful for comparing between investments. That is, a `$5` profit from a `$25` investment may be a decent gain, but the same `$5` profit from a `$10,000` investment would be trivial.

This is why the simple rate of return is calculated as the absolute gain or loss divided by the starting price of the investment. By expressing the return as a percentage of the originally invested amount, we can more easily compare across different investments.

Logarithmic Rate of Return

Another type of

Preview: Docs Loading link description
return
is the *logarithmic rate of return*, also known as the continuously compounded return. This is the expected return for an investment where the earnings are assumed to be continually reinvested over the time period. It is calculated by taking the difference between the log of the ending price and the log of the starting price.

$$r = log(E) - log(S) = log(S / E)$$

- `r`: logarithmic rate of return
- `S`: starting price of investment
- `E`: ending price of investment

The advantage of the log rate of return is that it is easy to make calculations about a single asset over time. On the other hand, calculating the simple rate of return is easier for dealing with multiple assets over the same time period.

We will take a closer look at both of these in the subsequent exercises. But whichever type of return we choose to use, it is important to remember to be consistent in using the same one for any further financial calculations.

Aggregate Across Time I

When describing the rate of return of an investment, something that is important to keep in mind is the time frame of the investment. For example, an investment with a `2%` rate of return over one day is surely not the same as an investment with a `2%` rate of return over one month. Thus, it is common to convert returns to a standard time period. Often, this means converting to the annual rate of return in a process called *annualizing*.

As we've covered earlier, aggregating across time for a single asset can easily be calculated using the log rate of return.

To convert a log rate of return from one time period to another, we can multiple the rate of return by the number of original time periods there are in the new time period.

$$R = r_0 * t$$

`r`: converted log rate of return
`r0`: original log rate of return
`t`: the number of original time periods in the new time period

For example, if we are converting daily returns to annual returns, `t` may be `252` because that is typically the number of trading days in a year. If we are converting monthly returns to annual returns, `t` would be `12`.

```
def display_as_percentage(val):
  return '{:.1f}%'.format(val * 100)

daily_return_a = 0.001
monthly_return_b = 0.022

# Write code here
print('The daily rate of return for Investment A is',
display_as_percentage(daily_return_a))

print('The monthly rate of return for Investment B is',
display_as_percentage(monthly_return_b))

def annualize_return(log_return, t):
  return log_return * t

annual_return_a = annualize_return(daily_return_a, 252)
print('The annual rate of return for Investment A is',
display_as_percentage(annual_return_a))

annual_return_b = annualize_return(monthly_return_b, 12)
print('The annual rate of return for Investment B is',
display_as_percentage(annual_return_b))
```

Aggregate Across Time II

Now, let's look at an extension of the previous conversion formula. Suppose we know the log rate of return for `5` days of a given year. Which daily log return would we use to calculate the annual return?

In this case, we can first take the average of the `5` daily log returns, then multiple by `252`, the number of trading days in a year. The general formula is:

r=...

`r`: converted log rate of return
r_{0n}: the nth log return from the original time period
`n`: the number of returns from the original time period
`t`: the number of original time periods in the new time period

Notice how if we only have one log return from the original time period, `n` equals to `1`, and we can simplify the equation to obtain the conversion formula we saw in the previous exercise.

On the other hand, if we know the log return for all `252` trading days, then `n` equals to `t`, and we will simply be summing up all the log returns across the new time period.

r=...

`r`: converted log rate of return
r_{0n}: the nth log return from the original time period
`t`: the number of original time periods in the new time period

```
def display_as_percentage(val):
  return '{:.1f}%'.format(val * 100)

daily_returns = [0.002, -0.002, 0.003, 0.002, -0.001]

# Write code here
def convert_returns(log_returns, t):
  return sum(log_returns) / len(log_returns) * t

annual_return = convert_returns(daily_returns, 252)

print('The annual rate of return is', display_as_percentage(annual_return))

weekly_return = convert_returns(daily_returns, 5)
weekly_return = sum(daily_returns)

print('The weekly rate of return is', display_as_percentage(weekly_return))
```

Variance

Now that we have a good understanding of rate of return, let's shift our focus to assessing the risk involved in an investment. One of the key statistics for understanding risk is variance. *Variance* is a measure of the spread of a dataset, or how far apart each value is from the mean. The greater the variance, the more spread out or variable the data is.

For example, let's take a look at the data below showing the returns of two investments over the course of a week. Which one has a higher variance? What do you think that tells us about their relative risk?

```
# Investment A
returns_a = [0.05, -0.10, -0.08, 0.05, 0.14]

# Investment B
returns_b = [-0.01, 0.02, 0.01, 0.04, 0.03]
```

Copy to Clipboard

As seen, the returns for Investment A fluctuates more throughout the week while the returns for Investment B remain relatively consistent. This means Investment A has a higher variance and therefore a higher risk. An asset with a high variance is generally a riskier one because its return can vary significantly in a short period of time, making it less stable and more unpredictable.

The formula for calculating variance is:

$$\sigma 2=$$

- $\sigma 2$: variance
- Xi: the ith value in the dataset
- x: the mean of the dataset
- n: the number of values in the dataset

```
import numpy as np

returns_disney = [0.22, 0.12, 0.01, 0.05, 0.04]
returns_cbs = [-0.13, -0.15, 0.31, -0.06, -0.29]

variance_disney = np.var(returns_disney)
variance_cbs = np.var(returns_cbs)

# Write code here
dataset = [10, 8, 9, 10, 12]
```

```
def calculate_variance(dataset):
  mean = sum(dataset) / len(dataset)

  numerator = 0
  for data in dataset:
    numerator += (data - mean) ** 2

  variance = numerator / len(dataset)

  return variance

variance_disney = calculate_variance(returns_disney)
variance_cbs = calculate_variance(returns_cbs)

print('The variance of Disney stock returns is', variance_disney)
print('The variance of CBS stock returns is', variance_cbs)
```

Standard Deviation

Although the variance is useful in determining the relative risk of an investment, it is sometimes not the easiest statistic to interpret since it does not have the same unit as the original data. As an alternative, it is common to use the standard deviation to describe the spread of the dataset.

Standard deviation is simply the square root of the variance. It has the same unit as the original dataset.

$$\sigma = (\sigma 2)\text{^}2$$

- σ: standard deviation
- x_i: the ith value in the dataset
- $\bar{x}$: the mean of the dataset
- n: the number of values in the dataset

```
from utils import calculate_variance
import numpy as np
# Import library here


def display_as_percentage(val):
  return '{:.1f}%'.format(val * 100)

returns_disney = [0.22, 0.12, 0.01, 0.05, 0.04]
returns_cbs = [-0.13, -0.15, 0.31, -0.06, -0.29]

stddev_disney = np.std(returns_disney)
stddev_cbs = np.std(returns_cbs)

# Write code here
dataset = [10, 8, 9, 10, 12]
```

```
print('The standard deviation of Disney stock returns is',
display_as_percentage(stddev_disney))
print('The standard deviation of CBS stock returns is', display_as_percentage(stddev_cbs))
```

Correlation I

Another important statistic for assessing risk is the correlation between the returns of two assets. *Correlation* is a measure of how closely two datasets are associated with each other. It is often represented by the correlation coefficient, which is a value that ranges between `-1` and `1`. This indicates whether there is a positive correlation, negative correlation, or no correlation:

Positive correlation – when the rate of
Preview: Docs Loading link description
return
of one asset deviates upward from its mean, the other usually deviates upward as well.
Negative correlation – when the rate of return of one asset deviates upward from its mean, the other usually deviates downward.
No correlation – when a change in one asset's rate of return does not dictate a change in another. The correlation coefficient will be close to `0`.

Two assets from the same industry generally have a positive correlation, as they are likely affected by similar external conditions. For example, automobile stocks may all be positively correlated with each other. Oil stocks, on the other hand, may be negatively correlated with automobile stocks because high oil costs can negatively impact car sales.

When building a portfolio, it is generally a good idea to include assets that are not correlated with each other. If assets are independent of one another, then there is a lower risk of the financial loss that can occur when assets in a portfolio are correlated. This allows for greater diversification and balances out the overall risk and return of the portfolio.

Note: You may also see correlation calculated using asset prices instead of returns, but returns are generally preferred. See this blogpost on correlation calculations for more information if you are interested!

```
from utils import calculate_correlation
import numpy as np

returns_general_motors = [0.018, -0.005, -0.047, -0.009, -0.012, 0.003, -0.027, -0.014,
0.029, -0.062, 0.009]
returns_ford = [0.002, -0.004, -0.027, -0.022, -0.001, 0.002, -0.006, -0.017, 0.035, -
0.029, 0.002]
returns_exxon_mobil = [0.008, 0.015, 0.009, 0.012, 0.003, -0.007, 0.006, 0.005, -0.048,
0.025, -0.012]
returns_apple = [-0.002, 0.007, -0.004, -0.004, 0.002, 0.013, -0.011, 0.017, -0.001, 0.012,
0.006]

corr_gm_ford = calculate_correlation(returns_general_motors, returns_ford)
print('The correlation coefficient between General Motors and Ford is', corr_gm_ford)

# Write code here
corr_gm_em = calculate_correlation(returns_general_motors, returns_exxon_mobil)
print('The correlation coefficient between General Motors and ExxonMobil is', corr_gm_em)

corr_gm_apple = calculate_correlation(returns_general_motors, returns_apple)
print('The correlation coefficient between General Motors and Apple is', corr_gm_apple)
```

```
corrcoef_matrix = np.corrcoef([returns_general_motors, returns_ford, returns_exxon_mobil,
returns_apple])
print(corrcoef_matrix)
```

Correlation II

Now that we have a good understanding of what correlation is and how to use it to assess the risk in an investment, let's take a closer look at how the correlation coefficient is calculated.

Below is the formula for the Pearson correlation coefficient:

$$r_{xy} = \frac{n * \Sigma(X_i * Y_i) - \Sigma X_i * \Sigma Y_i}{\sqrt{n * \Sigma X_i^2 - (\Sigma X_i)^2}\sqrt{n * \Sigma Y_i^2 - (\Sigma Y_i)^2}}$$

`rxy`: correlation coefficient
`Xi`: the ith value in dataset X
`Yi`: the ith value in dataset Y
`n`: the number of values in the dataset

```
from data import returns_general_motors, returns_ford, returns_exxon_mobil, returns_apple
from math import sqrt
import numpy as np

def calculate_correlation(set_x, set_y):
  # Sum of all values in each dataset
  sum_x = sum(set_x)
  sum_y = sum(set_y)

  # Sum of all squared values in each dataset
  sum_x2 = sum([x ** 2 for x in set_x])
  sum_y2 = sum([y ** 2 for y in set_y])

  # Sum of the product of each respective element in each dataset
  sum_xy = sum([x * y for x, y in zip(set_x, set_y)])

  # Length of dataset
  n = len(set_x)

  # Calculate correlation coefficient
  numerator = n * sum_xy - sum_x * sum_y
  denominator = sqrt((n * sum_x2 - sum_x ** 2) * (n * sum_y2 - sum_y ** 2))

  return numerator / denominator

# Function calls
print('The correlation coefficient between General Motors and Ford is',
calculate_correlation(returns_general_motors, returns_ford))
print('The correlation coefficient between General Motors and ExxonMobil is',
calculate_correlation(returns_general_motors, returns_exxon_mobil))
print('The correlation coefficient between General Motors and Apple is',
calculate_correlation(returns_general_motors, returns_apple))
```

Review

Congratulations on reaching the end!

In this lesson, we learned to calculate and understand the rate of

Preview: Docs Ends a function and sends a value back to the caller.
return
of an investment:

Simple Rate of Return – advantageous for aggregating over assets
Logarithmic Rate of Return – advantageous for aggregating over time

We also covered other key financial statistics and what they signify in terms of the risk of an investment:

Variance – measure of the spread of a dataset; an asset with low variance is less risky
Standard Deviation – square root of the variance; easier to interpret than variance because it has the same unit as the original dataset
Correlation – measure of the association between datasets; assets with no correlation have returns that are independent of each other

```
from utils import calculate_variance, calculate_stddev

def display_as_percentage(val):
  return '{:.1f}%'.format(val * 100)

annual_returns = [0.02, 0.05, -0.04, 0.04, 0.02, -0.02, 0.01, 0.03, 0.05, 0.02]

# Write code here
annual_returns_percentage = [display_as_percentage(r) for r in annual_returns]
print(annual_returns_percentage)

print('The historical annual rates of return are:', ', '.join(annual_returns_percentage))

variance = calculate_variance(annual_returns)
print('The variance of the rates of return is', variance)

stddev = display_as_percentage(calculate_stddev(annual_returns))
print('The standard deviation of the rates of return is', stddev)
```

Importing Data Using Datareader

Many financial institutions, stock markets, and world banks provide large amounts of the data they store to the public.

Most of this data is well organized, live updated, and accessible through the use of an application programming interface (API), which gives programming languages like Python a way to download and import it.

Pandas-Datareader Module

The **pandas-datareader** module is designed specifically to interact with some of the world's most popular finance data APIs, and import their data into an easily digestible pandas

Preview: Docs Loading link description
DataFrame
.
Each finance API is accessed using a different function exposed by pandas-datareader. Generally accessing each API requires a different set of arguments and information that needs to be provided by the programmer.

Throughout this lesson we'll be importing data from several of these APIs and playing around with it.

For a full list of all the data pandas-datareader is able to access you can check out the official documentation.

Instructions

Checkpoint 1 Passed

1.

In the text editor there is a small program which uses `pandas_datareader` to get gdp data from the World Bank API.

Take a moment to look it over. It's okay if it doesn't make sense right now, but notice some of the variables that are being created, start and end dates, a data indicator id and a list of countries.

Now run the code. It should print out the DataFrame of data that got imported from the World Bank.

https://pandas-datareader.readthedocs.io/en/latest/

```
import codecademylib3_seaborn

from pandas_datareader import wb

from datetime import datetime



start = datetime(2005, 1, 1)

end = datetime(2008, 1, 1)

indicator_id = 'NY.GDP.PCAP.KD'
```

```
gdp_per_capita = wb.download(indicator=indicator_id, start=start, end=end,
country=['US', 'CA', 'MX'])



print(gdp_per_capita)
```

Getting NASDAQ Symbols

The NASDAQ stock exchange identifies each of it's stocks using a unique symbol:

Apple - **APPL**
Google - **GOOGL**
Tesla - **TSLA**

It also provides a useful API for accessing the symbols that are currently trading on it.

Pandas-datareader provides several functions for importing data from NASDAQ's API through it's `nasdaq_trader` sub-module.

```
from pandas_datareader.nasdaq_trader import func
```

Copy to Clipboard
In the code above we're importing a function called `func` from the NASDAQ submodule.

In order to import the list of stock symbols, we'll want to use nasdaq_trader's `get_nasdaq_symbols` function.

```
symbols = get_nasdaq_symbols()
```

Copy to Clipboard
When called, it will go off to NASDAQ's API, and import the list of symbols trading at that moment.

The benefit of using pandas-datareader is that all of the logic for interacting with NASDAQ's API or any other API is encapsulated into easy to use sub-modules and functions like the ones above.

Instructions

Checkpoint 1 Enabled

1.

First we'll need to import the correct function from pandas_datareader. In the text editor write the code for importing the `get_nasdaq_symbols` function from the `pandas_datareader.nasdaq_trader` submodule.

```
from pandas_datareader.nasdaq_trader import [FUNC_NAME_HERE]
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now that we have the `get_nasdaq_symbols` function imported let's use it. Call the function (it doesn't take any arguments), store the resulting DataFrame in a variable called `symbols` and print it out.
This may take longer to run while the get_nasdaq_symbols() function is retrieving the NASDAQ symbols.

Filtering Data by Date

Many of the APIs pandas-datareader connects with allow us to filter the data we get back by time.

Financial institutions tend to keep track of data dating back several decades, and when we're importing that data, it's useful to be able to specify exactly when we want it to be from.

One API that does just that is the Federal Reserve Bank of St. Louis (FRED), which we can access by first importing the `pandas_datareader.data` sub-module and then calling it's `DataReader` function:

```
import pandas_datareader.data as web

web.DataReader('MORTGAGE30US', 'fred', start_date, end_date)
```

Copy to Clipboard
The `DataReader` function takes 4 arguments:

`'MORTGAGE30US'` - An identifier provided by the API specifying the data we want back, in this case 30 year mortgage data in the US
`'fred'` - The name of the API we want to access
`start_date`, `end_date` - The date range we want the data to be from

The start and end dates are special data types called `datetime`s, which can be created using the Python `datetime` module.

```
from datetime import datetime

start_date = datetime(2018, 7, 8) # year, month, day
end_date = datetime(2019, 4, 13)
```

Copy to Clipboard
More info on creating datetimes can be found in the documentation here.

By changing up the start and end `datetime`s, you can easily filter when the data you get back is from.

```
import codecademylib3_seaborn

from datetime import datetime

import pandas_datareader.data as web
```

```
start = datetime(2019, 1, 1)

end = datetime(2019, 2, 1)


sap_data = web.DataReader('SP500', 'fred', start, end)

print(sap_data)
```

Using the Shift Operation

```
import pandas_datareader.data as web
from datetime import datetime

start = datetime(2008, 1, 1)
end = datetime(2018, 1, 1)

gdp = web.DataReader('GDP', 'fred', start, end)
gdp['growth'] = gdp['GDP'].shift(-1)
print(gdp)
```

Calculating Basic Financial Statistics

Two useful calculations that can be made on financial data are variance and covariance.

To illustrate these concepts, let's use the example of a

Preview: Docs Loading link description
DataFrame
which measures stock and bond prices over time.

date	stocks	bonds
Jun	998	300
Jul	1230	275
Aug	1500	240
Sep	1750	190

Variance

Variance measures how far a set of numbers are spread out from their average. In finance, this is used to determine the volatility of investments.

```
dataframe['stocks'].var(); # 106427
dataframe['bonds'].var(); # 2272
```

Copy to Clipboard

In the variance calculations above, stocks have a larger value than bonds.

That's because the stock prices are more spread out than bonds, indicating that stocks are a more volatile investment.

Covariance

Covariance, in a financial context, describes the relationship between the returns on two different investments over a period of time, and can be used to help balance a portfolio.

Calling `cov()` on our stocks/bonds produces a matrix which defines the covariance values between each column pair in the DataFrame.

In our example data, when stock prices go up, bonds go down. We can use the covariance function to see this numerically.

```
dataframe.cov()
```

Copy to Clipboard

The above code produces the following output DataFrame:

type	stocks	bonds
stocks	106427	-15300
bonds	-15300	2272

Each value above represents the covariance between two columns.

The larger the number the more investments tend to move up and down at the same time.

The top right and bottom left columns represent the covariance between stocks and bonds. Here we get a negative number, indicating stocks and bonds tend to move in different directions.

Notice also the top left and bottom right columns are actually just the variances for stocks and bonds we saw earlier.

Review

Congrats! You are now ready to import your own financial data for analysis! Before you move on, let's take a minute to review what we've covered in this lesson.

Python is able to import financial data from csv files as well as public financial APIs.
The pandas `read_csv` function can be used to import data from a csv file into a pandas

Preview: Docs Loading link description
dataframe
.
Pandas-datareader makes it easy to import data from public financial APIs.
Python's datetime function can be used to create datetime objects which are often used to specify time ranges for financial data.
API keys are unique identifiers required for some APIs in order to access data.
Sometimes APIs can be flaky. To mitigate the damage this might cause it's best to test your code often and keep up to date with the pandas-datareader documentation and GitHub page.
The `shift` function can be used on the rows in a DataFrame column to shift them up or down.
Pandas provides common statistical functions like `var` and `cov` to make it easy to calculate variance and covariance on a dataset.

```
import codecademylib3_seaborn
import pandas as pd
import pandas_datareader.data as web
from datetime import datetime

apple_prices = pd.read_csv('apple_prices.csv')
print(apple_prices['open'].var())

start = datetime(2008, 1, 1)
end = datetime(2018, 1, 1)

gas_prices = web.DataReader('APUS12A74714', 'fred', start, end)
print(gas_prices)
```

Introduction to Visualizing Finance Data

Introduction to Visualizing Finance Data. Prepare to visualize.
Welcome to **Visualizing Finance Data**!

You have learned the basics of Python and have the skills you need to investigate and manipulate financial data. Now, it's time to learn how to visualize that data and gain deeper insights.

In this section, you'll be introduced to the most well-known Python module for graphing. We will be exploring how to use Matplotlib to create visually appealing charts and graphs that display data in engaging and informative ways.

By the end of the section, you will also learn how to perform mean-variance portfolio optimization and select a portfolio of assets that optimizes for expected returns and risk.

If you find yourself interested in exploring additional functionalities of Matplotlib as you complete the lessons in this section, check out the official Matplotlib Documentation.

Review

Now you've played around with several two-dimensional line plots in Matplotlib. You've seen how you can create simple, readable plots with few commands. You've also learned some commands to style and label your plots better. These are the concepts you've seen in Matplotlib so far:

- Creating a line graph from data
- Changing the appearance of the line
- Zooming in on different parts of the axis
- Putting labels on titles and axes
- Creating a more complex figure layout
- Adding legends to graphs
- Changing tick labels and positions
- Saving what you've made

Moving on, we'll learn how to make different kinds of plots (beyond line graphs!) in Matplotlib and how to choose between those plots when displaying data.

Let's do a final round of practice with all of the cool plotting concepts you've learned so far!

How to Select a Meaningful Visualization

This article will guide you through the process of selecting a graph for a visualization.

Brainstorming your visualization

The three steps in the data visualization process are *preparing*, *visualizing*, and *styling* data. When faced with a blank canvas, the second step of the process, visualizing the data, can be overwhelming. To help, we've created a diagram to guide the selection of a chart based on what you want to explore in your data.

When planning out a visualization, you'll usually have an idea of what questions you'll want to explore. However, you may initially wonder exactly which chart to use. This moment is one of the most exciting parts of the process!

During your brainstorming phase, you should consider two things:

The focusing question you want to answer with your chart
The type of data that you want to visualize

Depending on the focusing questions you're trying to answer, the type of chart you select should be different and intentional in its difference. In the diagram below, we have assigned Matplotlib visualizations to different categories. These categories explore common focusing questions and types of data you may want to display in a visualization.

A Diagram of Diagrams!

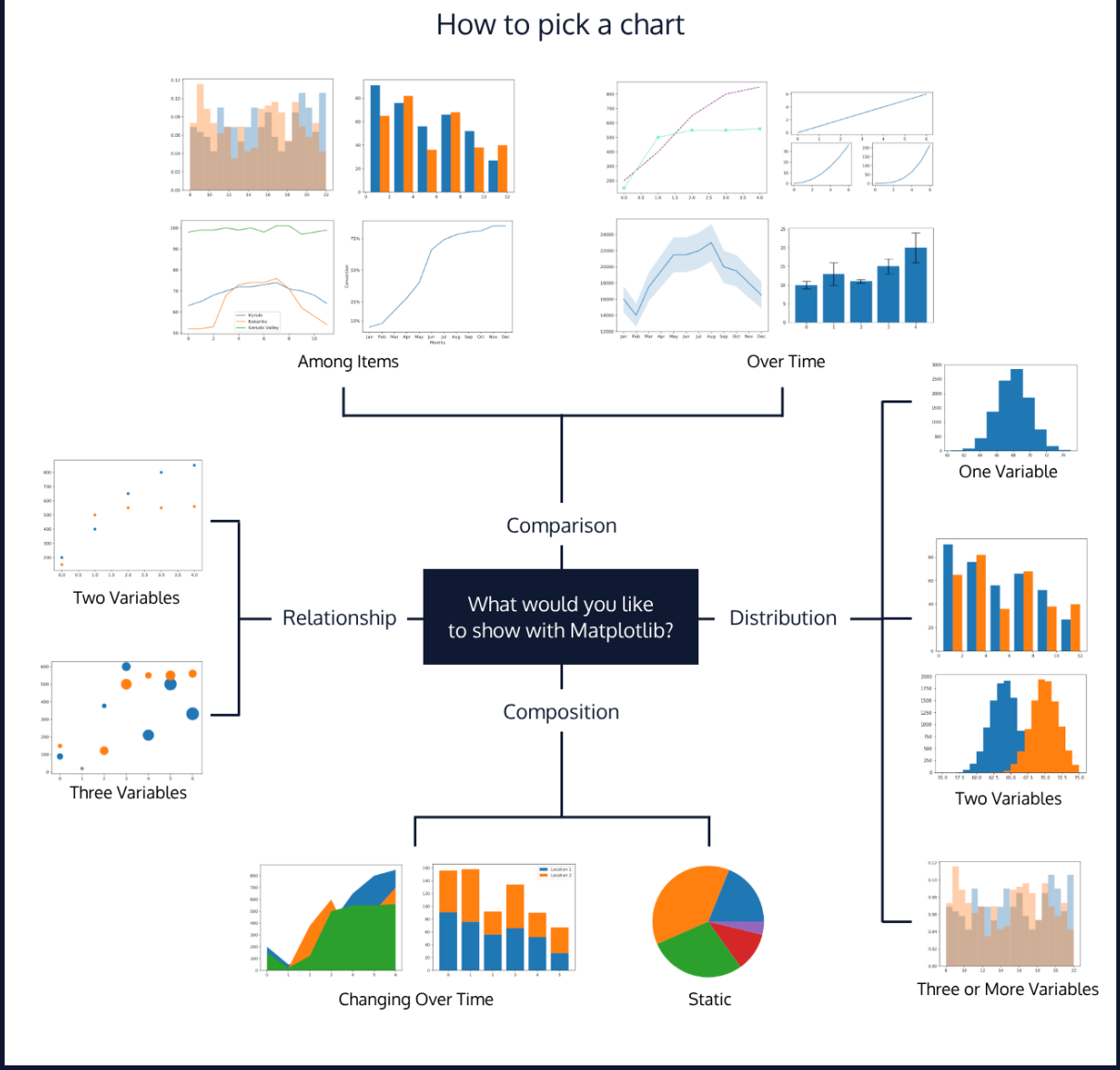


Chart categories

Composition charts

Focusing Question: What are the parts of some whole? What is the data made of?

Datasets that work well: Data pertaining to probabilities, proportions, and percentages can be visualized as with the graphs in this composition category. Charts in this category illustrate the different data components and their percentages as part of a whole.

Distribution Charts

Datasets that work well: Data in large quantities and/or with an array of attributes works well for these types of charts. Visualizations in this category will allow you to see patterns, re-occurrences, and a clustering of data points.

Note: In statistics, a commonly seen distribution is a bell curve, also known as a normal distribution. A bell curve is a bell-shaped distribution where most of the values in the dataset crowd around the average (also known as the mean), therefore causing the curve to form. If you want to see how values in the data are “distributed” across variables, the best way to do that would be with the visualizations in this category.

Relationship Charts

Focusing Question: How do variables relate to each other?

Datasets that work well: Data with two or more variables can be displayed in these charts. These charts typically illustrate a *correlation* between two or more variables. You can communicate this relationship by mapping multiple variables in the same chart. Correlation measures the strength of a relationship between variables.

Comparison Charts

Focusing Question: How do variables compare to each other?

Datasets that work well: Data must have multiple variables, and the visualizations in this category allow readers to compare those items against the others. For example, a line graph that has multiple lines, each belonging to a different variable. Multi-colored bar charts are also a great way to compare items in data.

Summary

When brainstorming a visualization, use the diagram above to guide the selection of your chart. Remember to be intentional in your selection by thinking about what type of data you’re dealing with and what focusing question you wish to answer.

Introduction

The fundamental goal of portfolio theory is to allocate your investments between multiple assets optimally. A Markowitz mean-variance optimization (MVO) is an approach to choosing how to allocate money between multiple assets by considering the risk-return trade-off of asset combinations.

The approach considers the expected returns and covariance for all assets. It returns a set of portfolios where:

The expected return is greater than any other with the same or lesser risk
The risk is less than any other with the same expected return

For example, a Markowitz optimization could recommend how to distribute your money between Nike, Under Armour, and Skechers. In this example, to make the recommendation, we will consider historical data from the last 22 quarters to split $10,000 between each company. To minimize the risk of your portfolio, you should invest the following amounts in each company:

Nike: $8,157
Under Armour: $1,667
Skechers: $176

You should invest the following amounts to maximize return:

Nike: $0
Under Armour: $0
Skechers: $10,000

Instructions

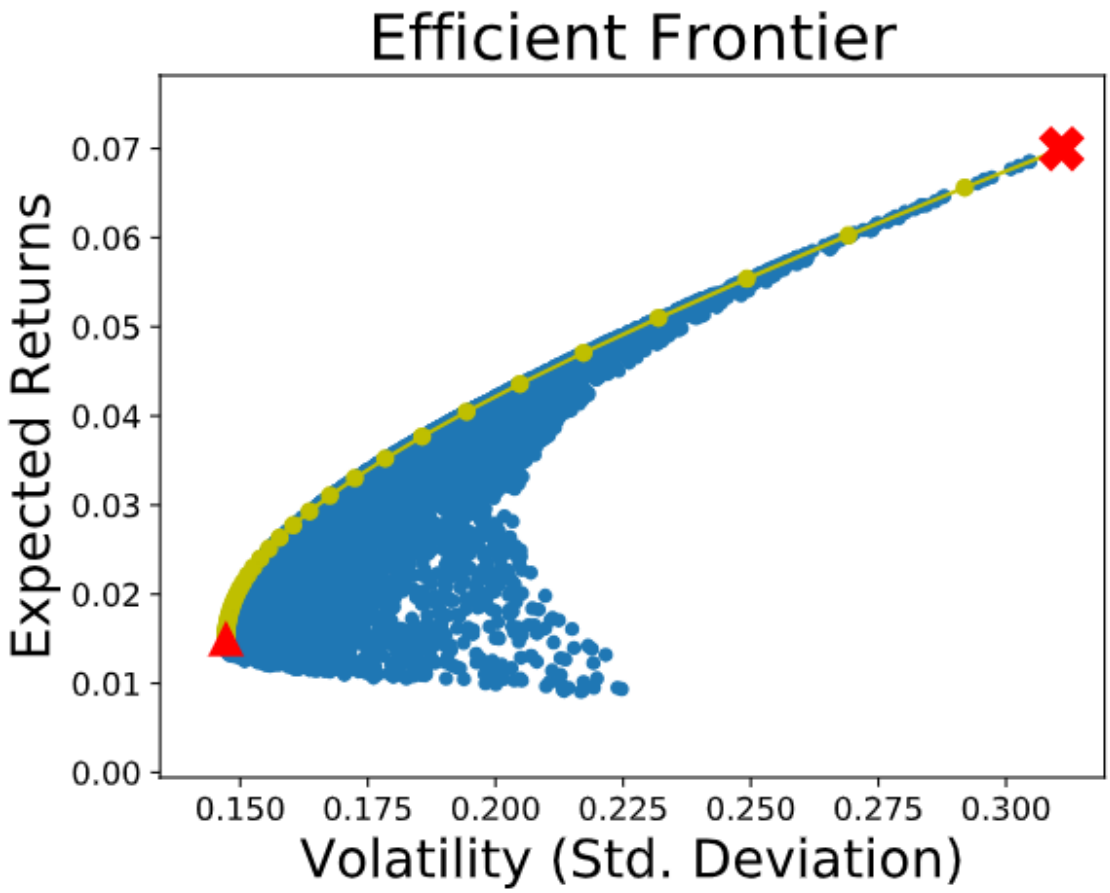
1. To the right, we display the risk and return for a collection of portfolios with Nike, Under Armour, and Skechers. The yellow line is called the efficient frontier — the set of portfolios that is optimized for risk and return.

   The red triangle marks the portfolio on the efficient frontier with the lowest risk (Example 1 above).
   The red X marks the portfolio on the efficient frontier with the highest return (Example 2 above).

It's okay if this figure is confusing to you. We'll go over it as we work our way through the lesson.

```
import pandas as pd
from matplotlib import pyplot as plt
#import codecademylib3_seaborn
import numpy as np
import cvxopt as opt
from cvxopt import blas, solvers

path = 'stock_data.csv'

stock_data = pd.read_csv(path)
selected = list(stock_data.columns[1:])

returns_quarterly = stock_data[selected].pct_change()
expected_returns = returns_quarterly.mean()
cov_quarterly = returns_quarterly.cov()

def optimal_portfolio(returns):
  n = returns.shape[1]
  returns = np.transpose(returns.as_matrix())

  N = 100
  mus = [10**(5.0 * t/N - 1.0) for t in range(N)]

  S = opt.matrix(np.cov(returns))
  pbar = opt.matrix(np.mean(returns, axis=1))

  G = -opt.matrix(np.eye(n))
  h = opt.matrix(0.0, (n ,1))
  A = opt.matrix(1.0, (1, n))
  b = opt.matrix(1.0)

  portfolios = [solvers.qp(mu*S, -pbar, G, h, A, b)['x']
                for mu in mus]

  returns = [blas.dot(pbar, x) for x in portfolios]
  risks = [np.sqrt(blas.dot(x, S*x)) for x in portfolios]

  m1 = np.polyfit(returns, risks, 2)
  x1 = np.sqrt(m1[2] / m1[0])

  wt = solvers.qp(opt.matrix(x1 * S), -pbar, G, h, A, b)['x']
  return np.asarray(wt), returns, risks


def return_portfolios(expected_returns, cov_matrix):
  port_returns = []
  port_volatility = []
  stock_weights = []

  num_assets = len(selected)
  num_portfolios = 5000

  for single_portfolio in range(num_portfolios):
      weights = np.random.random(num_assets)
      weights /= np.sum(weights)
      returns = np.dot(weights, expected_returns)
```

```
        volatility = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
        port_returns.append(returns)
        port_volatility.append(volatility)
        stock_weights.append(weights)

    portfolio = {'Returns': port_returns,
                 'Volatility': port_volatility}

    for counter,symbol in enumerate(selected):
        portfolio[symbol +' Weight'] = [Weight[counter] for Weight in stock_weights]

    df = pd.DataFrame(portfolio)

    column_order = ['Returns', 'Volatility'] + [stock+' Weight' for stock in selected]

    df = df[column_order]

    return df

df = return_portfolios(expected_returns, cov_quarterly)

weights, returns, risks = optimal_portfolio(returns_quarterly[1:])

min_risk_idx = np.array(risks).argmin()
max_return_idx = np.array(risks).argmax()
min_risk = [risks[min_risk_idx], returns[min_risk_idx]]
max_return = [risks[max_return_idx], returns[max_return_idx]]

df.plot.scatter(x='Volatility', y='Returns', fontsize=12)
plt.plot(risks, returns, 'y-o')
plt.plot(min_risk[0],min_risk[1],'r^', markersize=12)
plt.plot(max_return[0],max_return[1],'rX',markersize=16)
plt.ylabel('Expected Returns',fontsize=20)
plt.xlabel('Volatility (Std. Deviation)',fontsize=20)
plt.title('Efficient Frontier', fontsize=24)
plt.show()
```



LINEARREGRESSION

Points and Lines

In the last exercise, you were probably able to make a rough estimate about the next data point for Sandra's lemonade stand without thinking too hard about it. For our program to make the same level of guess, we have to determine what a line would look like through those data points.

A line is determined by its *slope* and its *intercept*. In other words, for each point `y` on a line we can say:

$$y=mx+b$$

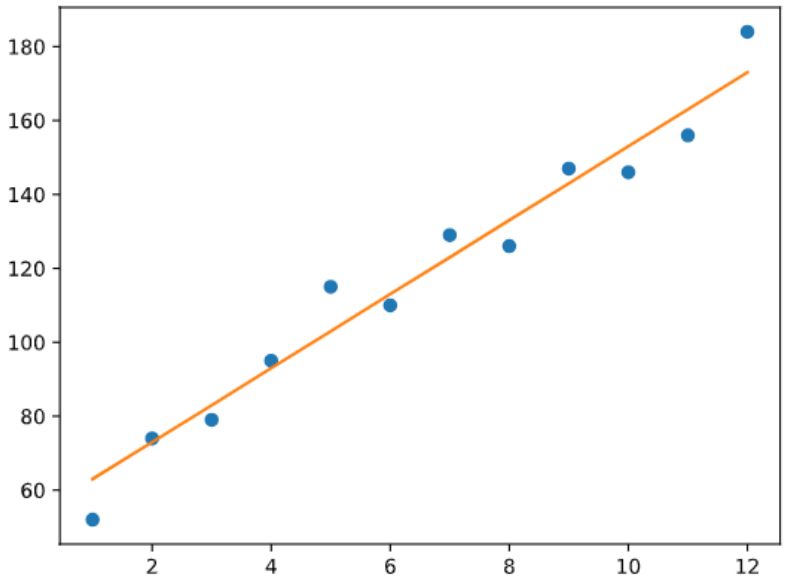
$$y=mx+b$$

where `m` is the slope, and `b` is the intercept. `y` is a given point on the y-axis, and it corresponds to a given `x` on the x-axis.

The slope is a measure of how steep the line is, while the intercept is a measure of where the line hits the y-axis.

When we perform Linear Regression, the goal is to get the “best” `m` and `b` for our data. We will determine what “best” means in the next exercises.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
revenue = [52, 74, 79, 95, 115, 110, 129, 126, 147, 146, 156, 184]

#slope:
m = 10
#intercept:
b = 53

y = [m*x + b for x in months]

plt.plot(months, revenue, "o")
plt.plot(months, y)

plt.show()
```

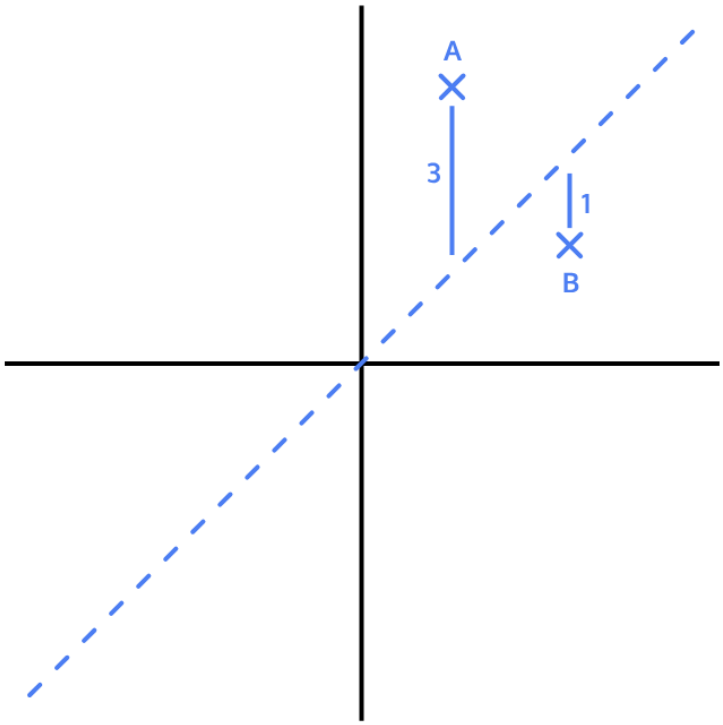


Loss

When we think about how we can assign a slope and intercept to fit a set of points, we have to define what the *best fit* is.

For each data point, we calculate **loss**, a number that measures how bad the model’s (in this case, the line’s) prediction was. You may have seen this being referred to as error.

We can think about loss as the squared distance from the point to the line. We do the squared distance (instead of just the distance) so that points above and below the line both contribute to total loss in the same way:



In this example:

For point A, the squared distance is `9` (3^2)
For point B, the squared distance is `1` (1^2)

So the total loss, with this model, is `10`. If we found a line that had less loss than `10`, that line would be a better model for this data.

```
x = [1, 2, 3]
y = [5, 1, 3]

#y = x
m1 = 1
b1 = 0

#y = 0.5x + 1
m2 = 0.5
b2 = 1

y_predicted1 = [m1*x_val + b1 for x_val in x]
y_predicted2 = [m2*x_val + b2 for x_val in x]

total_loss1 = 0
total_loss2 = 0

for i in range(len(y)):
  total_loss1 += (y[i] - y_predicted1[i])**2
  total_loss2 += (y[i] - y_predicted2[i])**2

print(total_loss1, total_loss2)
better_fit = 2
```

Minimizing Loss

The goal of a linear regression model is to find the slope and intercept pair that minimizes loss on average across all of the data.

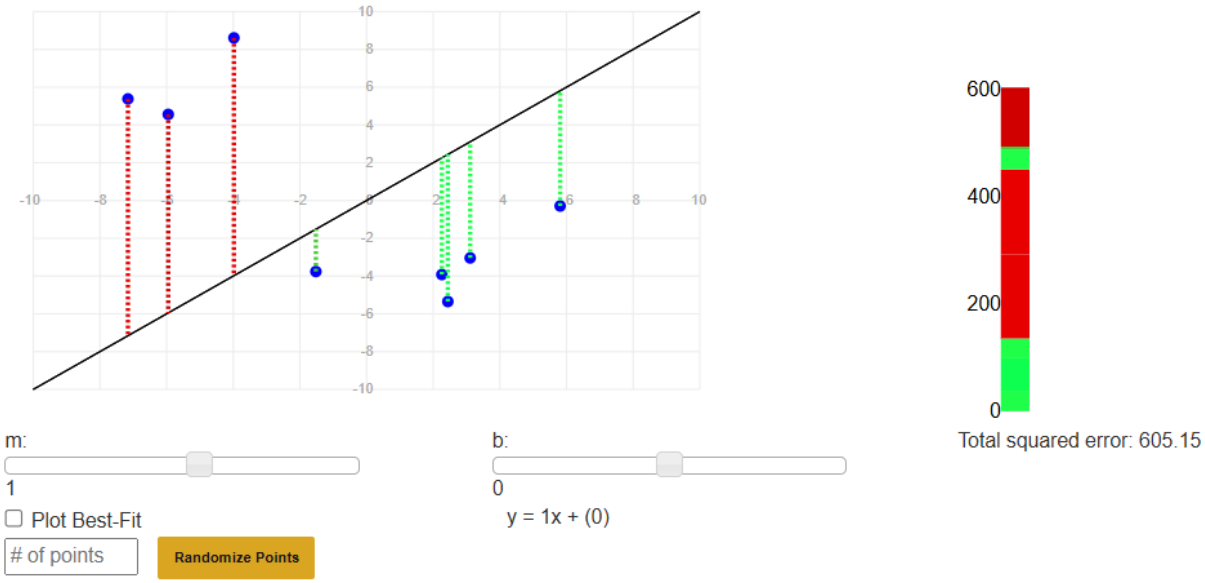
The interactive visualization in the browser lets you try to find the line of best fit for a random set of data points:

The slider on the left controls the `m` (slope)
The slider on the right controls the `b` (intercept)

You can see the total loss on the right side of the visualization. To get the line of best fit, we want this loss to be as small as possible.

To check if you got the best line, check the “Plot Best-Fit” box.

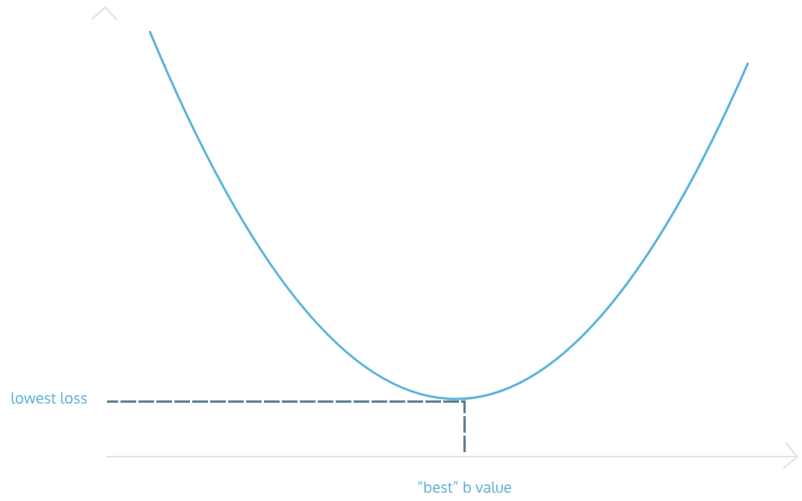
Randomize a new set of points and try to fit a new line by entering the number of points you want (try `8`!) in the textbox and pressing Randomize Points.



Gradient Descent for Intercept

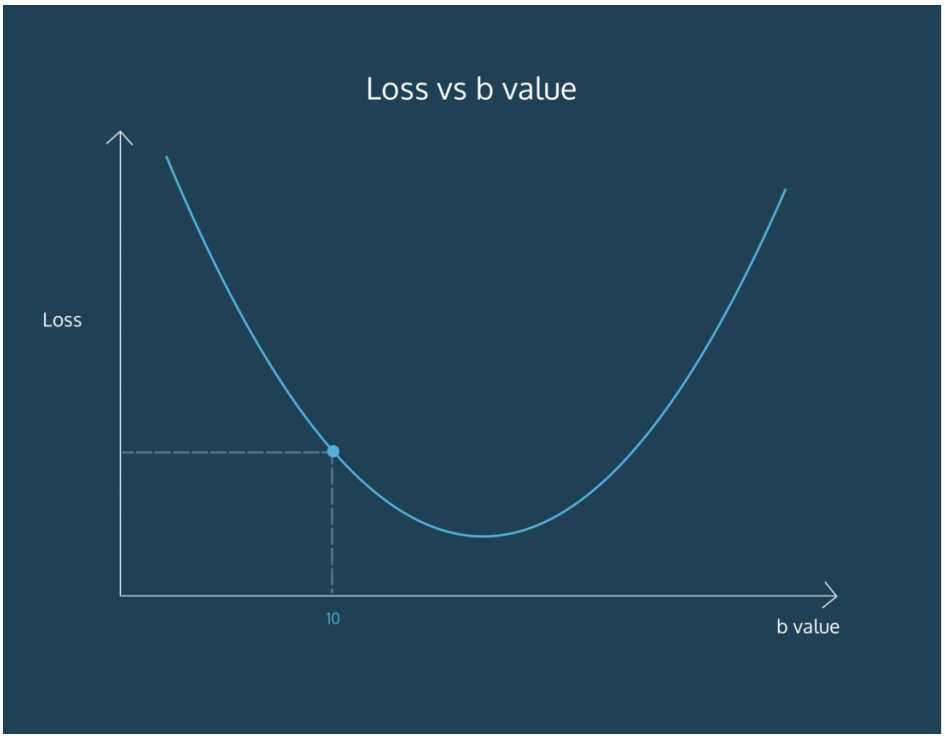
As we try to minimize loss, we take each

Preview: Docs Loading link description
[parameter]
we are changing, and move it as long as we are decreasing loss. It’s like we are moving down a hill, and stop once we reach the bottom:



The process by which we do this is called **gradient descent**. We move in the direction that decreases our loss the most. *Gradient* refers to the slope of the curve at any point.

For example, let's say we are trying to find the intercept for a line. We currently have a guess of `10` for the intercept. At the point of `10` on the curve, the slope is downward. Therefore, if we increase the intercept, we should be lowering the loss. So we follow the gradient downwards.



We derive these gradients using calculus. It is not crucial to understand how we arrive at the gradient equation. To find the gradient of loss as intercept changes, the formula comes out to be:

$$-\frac{2}{N}\sum_{i=1}^{N}(y_i - (mx_i + b))$$

`N` is the number of points we have in our dataset
`m` is the current gradient guess
`b` is the current intercept guess

Basically:

we find the sum of `y_value - (m*x_value + b)` for all the `y_value`s and `x_value`s we have and then we multiply the sum by a factor of `-2/N`. `N` is the number of points we have.

```
def get_gradient_at_b(x, y, m, b):
    diff = 0
    N = len(x)
    for i in range(0, len(x)):
      y_val = y[i]
      x_val = x[i]
      diff += (y_val - ((m * x_val) + b))

    b_gradient = -2/N * diff
    return b_gradient
```

Gradient Descent for Slope

We have a function to find the gradient of `b` at every point. To find the `m` gradient, or the way the loss changes as the slope of our line changes, we can use this formula:

$$-\frac{2}{N}\sum_{i=1}^{N}(y_i - (mx_i + b))$$

Once more:

`N` is the number of points you have in your dataset
`m` is the current gradient guess
`b` is the current intercept guess

To find the `m` gradient:

we find the sum of `x_value * (y_value - (m*x_value + b))` for all the `y_value`s and `x_value`s we have and then we multiply the sum by a factor of `-2/N`. `N` is the number of points we have.

Once we have a way to calculate both the `m` gradient and the `b` gradient, we'll be able to follow both of those gradients downwards to the point of lowest loss for both the `m` value and the `b` value. Then, we'll have the best `m` and the best `b` to fit our data!

```
def get_gradient_at_b(x, y, m, b):
    diff = 0
    N = len(x)
    for i in range(N):
      y_val = y[i]
      x_val = x[i]
      diff += (y_val - ((m * x_val) + b))
    b_gradient = -2/N * diff
    return b_gradient

def get_gradient_at_m(x, y, m, b):
    diff = 0
    N = len(x)
    for i in range(N):
      y_val = y[i]
      x_val = x[i]
      diff += x_val*(y_val - ((m * x_val) + b))
    m_gradient = -2/N * diff
    return m_gradient
```

Put it Together

Now that we know how to calculate the gradient, we want to take a "step" in that direction. However, it's important to think about whether that step is too big or too small. We don't want to overshoot the minimum error!

We can scale the size of the step by multiplying the gradient by a *learning rate*.

To find a new `b` value, we would say:

```
new_b = current_b - (learning_rate * b_gradient)
```

Copy to Clipboard

where `current_b` is our guess for what the `b` value is, `b_gradient` is the gradient of the loss curve at our current guess, and `learning_rate` is proportional to the size of the step we want to take.

In a few exercises, we'll talk about the implications of a large or small learning rate, but for now, let's use a fairly small value.

```
def get_gradient_at_b(x, y, b, m):
  N = len(x)
  diff = 0
  for i in range(N):
    x_val = x[i]
    y_val = y[i]
    diff += (y_val - ((m * x_val) + b))
  b_gradient = -(2/N) * diff
  return b_gradient

def get_gradient_at_m(x, y, b, m):
  N = len(x)
  diff = 0
  for i in range(N):
      x_val = x[i]
      y_val = y[i]
      difaf += x_val * (y_val - ((m * x_val) + b))
  m_gradient = -(2/N) * diff
  return m_gradient
```

```
#Your step_gradient function here
def step_gradient(x, y, b_current, m_current):
    b_gradient = get_gradient_at_b(x, y, b_current, m_current)
    m_gradient = get_gradient_at_m(x, y, b_current, m_current)
    b = b_current - (0.01 * b_gradient)
    m = m_current - (0.01 * m_gradient)
    return [b, m]

months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
revenue = [52, 74, 79, 95, 115, 110, 129, 126, 147, 146, 156, 184]

# current intercept guess:
b = 0
# current slope guess:
m = 0

b, m = step_gradient(months, revenue, b, m)
print(b, m)
```

Convergence

How do we know when we should stop changing the parameters `m` and `b`? How will we know when our program has learned enough?

To answer this, we have to define convergence. **Convergence** is when the loss stops changing (or changes very slowly) when parameters are changed.

Hopefully, the
Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.
algorithm
will converge at the best values for the parameters `m` and `b`.

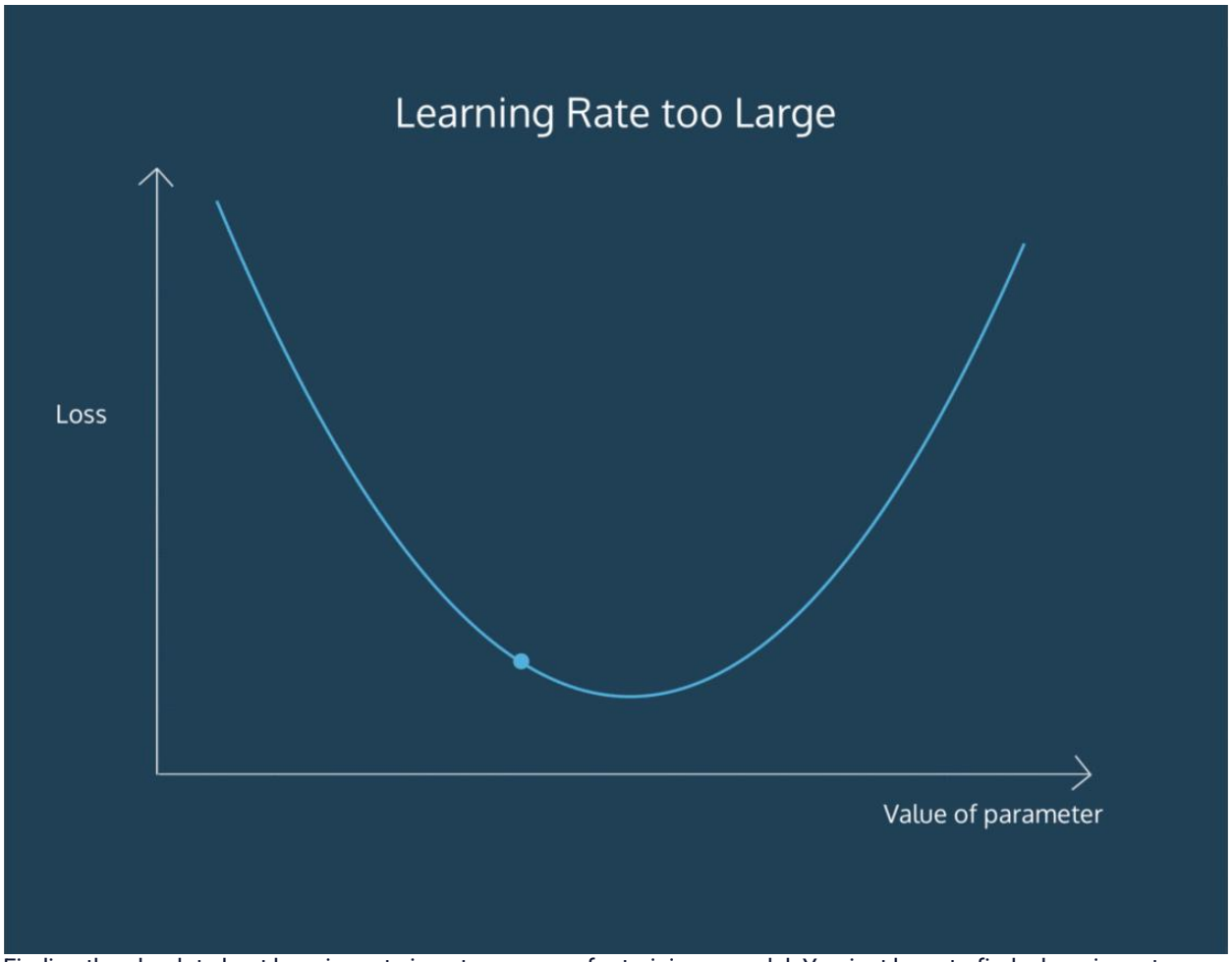
```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
from data import bs

iterations = range(1400)

plt.plot(iterations, bs)
plt.xlabel("Iterations")
plt.ylabel("b value")
plt.show()

convergence_b = 47
num_iterations = 800
```

Learning Rate

We want our program to be able to iteratively *learn* what the best `m` and `b` values are. So for each `m` and `b` pair that we guess, we want to move them in the direction of the gradients we've calculated. But how far do we move in that direction?

We have to choose a **learning rate**, which will determine how far down the loss curve we go.

A small learning rate will take a long time to converge — you might run out of time or cycles before getting an answer. A large learning rate might skip over the best value. It might *never* converge! Oh no!



Finding the absolute best learning rate is not necessary for training a model. You just have to find a learning rate large enough that gradient descent converges with the efficiency you need, and not so large that convergence never happens.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
from data import bs, bs_000000001, bs_01


# iterations = range(1400)
iterations = range(100)


plt.plot(iterations, bs_01)
plt.xlabel("Iterations")
plt.ylabel("b value")
plt.show()
```

Put it Together II

At each step, we know how to calculate the gradient and move in that direction with a step size proportional to our learning rate. Now, we want to make these steps until we reach convergence.

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt

def get_gradient_at_b(x, y, b, m):
  N = len(x)
  diff = 0
  for i in range(N):
    x_val = x[i]
    y_val = y[i]
    diff += (y_val - ((m * x_val) + b))
  b_gradient = -(2/N) * diff
  return b_gradient

def get gradient at m(x, y, b, m):
  N = len(x)
  diff = 0
  for i in range(N):
      x_val = x[i]
      y_val = y[i]
      diff += x_val * (y_val - ((m * x_val) + b))
  m_gradient = -(2/N) * diff
  return m gradient

#Your step_gradient function here
def step_gradient(b_current, m_current, x, y, learning_rate):
    b gradient = get gradient at b(x, y, b current, m current)
    m_gradient = get_gradient_at_m(x, y, b_current, m_current)
    b = b_current - (learning_rate * b_gradient)
    m = m current - (learning rate * m gradient)
    return [b, m]

#Your gradient_descent function here:
def gradient_descent(x, y, learning_rate, num_iterations):
  b = 0
  m = 0
  for i in range(num_iterations):
    b, m = step gradient(b, m, x, y, learning rate)
  return [b,m]

months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
revenue = [52, 74, 79, 95, 115, 110, 129, 126, 147, 146, 156, 184]

#Uncomment the line below to run your gradient_descent function
b, m = gradient_descent(months, revenue, 0.01, 1000)

#Uncomment the lines below to see the line you've settled upon!
y = [m*x + b for x in months]

plt.plot(months, revenue, "o")
plt.plot(months, y)

plt.show()
```

Scikit-Learn

Preview: Docs Loading link description
algorithm
 from scratch.
Luckily, we don't have to do this every time we want to use linear regression. We can use Python's scikit-learn library. Scikit-learn, or `sklearn`, is used specifically for

Preview: Docs Loading link description
Machine Learning
. Inside the `linear_model` module, there is a `LinearRegression()` function we can use:

```
from sklearn.linear_model import LinearRegression
```

Copy to Clipboard
You can first create a `LinearRegression` model, and then fit it to your `x` and `y` data:

```
line_fitter = LinearRegression()
line_fitter.fit(X, y)
```

Copy to Clipboard
The `.fit()`

Preview: Docs Loading link description
method
 gives the model two
Preview: Docs Loading link description
variables
 that are useful to us:

- the `line_fitter.coef_`, which contains the slope
- the `line_fitter.intercept_`, which contains the intercept

We can also use the `.predict()` function to pass in x-values and receive the y-values that this line would predict:

```
y_predicted = line_fitter.predict(X)
```

Copy to Clipboard
Note: the `num_iterations` and the `learning_rate` that you learned about in your own implementation have default values within scikit-learn, so you don't need to worry about setting them specifically!

```
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import numpy as np

temperature = np.array(range(60, 100, 2))
temperature = temperature.reshape(-1, 1)
sales = [65, 58, 46, 45, 44, 42, 40, 40, 36, 38, 38, 28, 30, 22, 27, 25, 25, 20, 15, 5]

plt.plot(temperature, sales, 'o')

line_fitter = LinearRegression()
line_fitter.fit(temperature, sales)
sales_predict = line_fitter.predict(temperature)

plt.plot(temperature, sales_predict)
plt.show()
```

Review

We have seen how to implement a linear regression

Preview: Docs Loading link description
algorithm
in Python, and how to use the linear regression model from scikit-learn. We learned:

- We can measure how well a line fits by measuring *loss*.
- The goal of linear regression is to minimize loss.
- To find the line of best fit, we try to find the `b` value (intercept) and the `m` value (slope) that minimize loss.
- *Convergence* refers to when the parameters stop changing with each iteration.
- *Learning rate* refers to how much the parameters are changed on each iteration.
- We can use Scikit-learn's `LinearRegression()` model to perform linear regression on a set of points.

Analyze Financial Data with Python Capstone

Capstone project for the Analyze Financial Data with Python Skill Path.

You are working as a wealth manager at a small firm where you have clients seeking advice on how to invest their money. A young client wants to invest a large amount of their savings in a portfolio of stocks, but they are unsure of what stocks to invest in and at what amounts.

In this capstone project, you will create a presentation that showcases a portfolio of stocks that you recommend your client invest in, and provide the weights at which they should invest in each stock depending on their risk preference.

The purpose of this capstone is to practice accessing and downloading financial data, calculating financial statistics to understand the individual performance of an asset and how groups of assets are related, and how to allocate investments across assets for optimal performance

Compared to the other projects you have completed thus far, we are requiring few restrictions on how you structure your code. The project is much more open-ended, and you should use your

creativity. In addition, much of the code you write for later parts of this project will depend on how you decide to implement earlier parts. **Therefore, we strongly encourage you to read through the entire assignment before writing any code.**

Import/Download the Data

The financial data you use in this project is up to you. You can access financial data using one of the APIs accessed through the pandas-datareader package or from websites such as Yahoo Finance. Make sure to find data for at least, but not limited to, four (4) stocks to include in your analysis.

Load the data into a pandas DataFrame so you can easily view and manipulate the data.

Calculate Financial Statistics

Calculate some of the financial statistics you have learned about to gain insights into the stocks and how they relate to each other. What are the returns of the stocks over different time periods? How risky are each of the stocks when compared to each other? Do the returns of the stocks correlate with each other, or are they diversified?

Optimized Portfolio

Perform a mean-variance portfolio optimization that shows the efficient frontier for the group of stocks you have selected. If the investor is less risky, how should she allocate her funds across the portfolio? If she is more risky, how should she allocate her funds? Indicate multiple investment options at different risk levels and specify the returns.

Create Your Presentation

We want to see:

Share your Presentation

Codecademy forum

Good luck!